

Semestrální práce dokumentace

Výpočet determinantu čtvercové matice pomocí Gaussovy eliminace (sekvenčně a paralelně)

Determinant

Determinant je základní vlastnost čtvercové matice, která se často používá např. pro ověření invertibility matice ($\det(A) \neq 0$).

U velkých matic ale hodnota $\det(A)$ velmi rychle přeteče i pro typ long double, proto program nevypisuje determinant přímo, ale ve tvaru:

- $\text{sign} \in \{-1, 0, 1\}$ – znaménko determinantu,
- $\log_{10}(|\det|)$ – logaritmus absolutní hodnoty determinantu.

Gaussova eliminace

Algoritmus postupně převádí matici na horní trojúhelníkový tvar pomocí elementárních úprav řádků.

Pro stabilitu se v každém sloupci vybírá řádek s maximální hodnotou $|A(i,k)|$ (částečná volba hlavního prvku).

Po převodu na horní trojúhelníkový tvar platí:

- Determinant získáme jako součin prvků na hlavní diagonále horní trojúhelníkové matice. Pokud během výpočtu dojde k prohození dvou řádků, změní se znaménko determinantu (za každé prohození se výsledek násobí -1).
- v programu se místo součinu počítá $\log_{10}(|\det|)$ jako součet $\log_{10}(|A(i,i)|)$.

Kroky a jak funguje algoritmus

1. V aktuálním sloupci k vybereme řádek s největší absolutní hodnotou v daném sloupci (hlavní prvek pro tento krok).
2. Pokud je vybraný diagonální prvek „moc malý“ ($|A(k,k)| < \text{eps}$), pak $\det(A) = 0$.
3. Pokud vybraný řádek není k , řádky se prohodí a tím se změní znaménko determinantu.
4. Vynulují se prvky pod diagonálou ve sloupci k (řádková eliminace).
5. Na konci se spočítá znaménko a $\log_{10}(|\det|)$ z diagonály horní trojúhelníkové matice.

Paralelní verze algoritmu

Paralelní varianta používá stejný postup volby hlavního prvku jako sekvenční verze.

Rozdíl je pouze ve kroku eliminace: řádky pod aktuálním řádkem (tj. pod diagonálou) jsou rozděleny do bloků a zpracovány ve více vláknech (`std::thread`).

- Počet vláken lze nastavit parametrem `--threads N`.
- Pokud není zadán, použije se `std::thread::hardware_concurrency()`.
- V každém kroku k se vlákna vytvoří, zpracují svůj blok řádků a následně se čeká na `join()`.

Tento přístup funguje dobře pro velké matice, ale pro malé vstupy může být pomalejší kvůli režii vytváření vláken.

Popis implementace a tříd

Program je rozdělen do tří hlavních tříd:

1) Třída Matrix

- `size()`
Vrací rozměr matice.
- `operator()(i, j)`
Přístup k prvkům matice (uložení v 1D poli).
- `swapRows(r1, r2)`
Prohodí dva řádky (využívá se, když vybraný řádek s hlavním prvkem není na pozici k).
- `readFrom(std::istream& in)` Načte matici ze souboru nebo `stdin`. Očekávaný formát:

```
n
a11 a12 ... a1n
```

```
...
an1 an2 ... ann
```

2) Třída DeterminantCalculator

Obsahuje dvě implementace výpočtu determinantu:

- `computeSequential(Matrix a)`
Sekvenční Gaussova eliminace s částečnou volbou hlavního prvku.
- `computeParallel(Matrix a, int num_threads)`
Paralelní Gaussova eliminace – eliminace řádků v kroku k je rozdělena mezi vlákna.

Výsledkem obou metod je struktura:

- `DetResult { int sign; long double logAbs; }`

3) Třída Application

Zajišťuje práci s argumenty, načtení matice a vypsání výsledků.

- `parseArgs(argc, argv)`
Zpracuje parametry příkazové řádky.
- `run()`
Načte matici, spustí zvolený algoritmus a změří čas.
- `printResult(tag, DetResult)` Vypíše výsledek ve formátu `sign a log10(|det|)`.

Ovládání aplikace

Program přijímá následující argumenty:

Přepínač	Popis
<code>--help</code>	Zobrazí nápovědu a skončí
<code>--algo MODE</code>	<code>seq</code> <code>par</code> <code>both</code>
<code>--threads N</code>	Počet vláken pro paralelní běh (default: HW concurrency)
<code>--input FILE</code>	Načte matici ze souboru

Pozn.: Pokud není použit `--input`, lze soubor předat i jako poslední argument (nepovinně), jinak se čte ze stdin.

Příklad spuštění

Sekvenční výpočet:

```
./algorithm --algo seq --input ../data/matrix2000.txt
```

Paralelní výpočet se 8 vlákny:

```
./algorithm --algo par --threads 8 --input ../data/matrix2000.txt
```

Porovnání obou verzí na stejném vstupu:

```
./algorithm --algo both --input ../data/matrix5000.txt
```

Testování programu

Testování bylo provedeno na 6-ti vstupech z adresáře data/.

Ve všech případech se porovnávaly výsledky sekvenční a paralelní verze pomocí rozdílu $\log_{10}(|\det|)$.

1) Malá matice 3×3 (matrix3.txt)

```
zhmykh@zhmykh:~/CLionProjects/algorithm/cmake-build-release$ ./algorithm --algo both --input ../data/matrix3.txt
Determinant (seq): sign=-1, log10(|det|)=2.2201080880
Time (seq): 0 ms

Determinant (par): sign=-1, log10(|det|)=2.2201080880
Time (par): 0 ms

Absolute difference between log10(|det|): 0.0000000000
```

2) Matice 50×50 (matrix50.txt)

U malých matic se projevuje režie vláken – paralelní verze může být pomalejší.

```
zhmykh@zhmykh:~/CLionProjects/algorithm/cmake-build-release$ ./algorithm --algo both --input ../data/matrix50.txt
Determinant (seq): sign=-1, log10(|det|)=106.6640579103
Time (seq): 0 ms

Determinant (par): sign=-1, log10(|det|)=106.6640579103
Time (par): 11 ms

Absolute difference between log10(|det|): 0.0000000000
```

3) Matice 1000×1000 (matrix_1000.txt)

```
zhmykh@zhmykh:~/CLionProjects/algorithm/cmake-build-release$ ./algorithm --algo both --input ../data/matrix_1000.txt
Determinant (seq): sign=-1, log10(|det|)=2064.5349710636
Time (seq): 931 ms

Determinant (par): sign=-1, log10(|det|)=2064.5349710636
Time (par): 447 ms

Absolute difference between log10(|det|): 0.0000000000
```

4) Matice 1000×1000 – větší hodnoty (matrix1000_big.txt)

```
zhmykh@zhmykh:~/CLionProjects/algorithm/cmake-build-release$ ./algorithm --algo both --input ../data/matrix1000_big.txt
Determinant (seq): sign=1, log10(|det|)=5042.7204233788
Time (seq): 937 ms

Determinant (par): sign=1, log10(|det|)=5042.7204233788
Time (par): 453 ms

Absolute difference between log10(|det|): 0.0000000000
```

5) Matice 2000×2000 (matrix2000.txt)

```
zhmykh@zhmykh:~/CLionProjects/algorithm/cmake-build-release$ ./algorithm --algo both --input ../data/matrix2000.txt
Determinant (seq): sign=-1, log10(|det|)=4430.9241345924
Time (seq): 7405 ms

Determinant (par): sign=-1, log10(|det|)=4430.9241345924
Time (par): 5267 ms

Absolute difference between log10(|det|): 0.0000000000
```

6) Velká matice 5000×5000 (matrix5000.txt)

```
zhmykh@zhmykh:~/CLionProjects/algorithm/cmake-build-release$ ./algorithm --algo both --input ../data/matrix5000.txt
Determinant (seq): sign=1, log10(|det|)=12071.9404804411
Time (seq): 113800 ms

Determinant (par): sign=1, log10(|det|)=12071.9404804411
Time (par): 67039 ms

Absolute difference between log10(|det|): 0.0000000000
```

Shrnutí časů

Vstup	Rozměr	Čas (seq)	Čas (par)	Zrychlení
matrix3.txt	3	0 ms	0 ms	—
matrix50.txt	50	0 ms	11 ms	—
matrix_1000.txt	1000	931 ms	447 ms	~2.08×
matrix1000_big.txt	1000	937 ms	453 ms	~2.07×
matrix2000.txt	2000	7405 ms	5267 ms	~1.41×
matrix5000.txt	5000	113800 ms	67039 ms	~1.70×

Závěr

Na základě provedených testů lze konstatovat, že obě implementace počítají determinant korektně – pro všechny testované vstupy se shoduje znaménko i $\log_{10}(|\det|)$ (rozdíl byl 0).

Sekvenční varianta je jednoduchá a pro malé matice velmi rychlá (bez režie vláken). Paralelní varianta přináší znatelné zrychlení u větších matic (např. kolem 2× pro rozměr 1000×1000), ale škálování není lineární – výkon je omezen zejména pamětovou propustností a režii vytváření vláken v každém kroku eliminace.

Použité zdroje

Základní teorie determinantu a Gaussovy eliminace byla čerpána ze studijních materiálů k lineární algebře. cppreference.com – oficiální referenční dokumentace jazyka C++ (std::vector, std::thread, I/O, <chrono>).

Při implementaci byly použity výhradně standardní knihovny jazyka C++:

- <iostream>
- <vector>
- <fstream>
- <string>
- <thread>
- <chrono>
- <stdexcept>
- <cmath>

Žádné externí knihovny nebyly použity.